Note: where an exercise involves more than one class, those classes may be split across multiple files (or organized into packages) or kept together in a single `.java` file — either approach is fine throughout this document.

Exercise 1 — Car Registry with Serialization

`Car` class

- Fields: `carModel` (String), `make` (String), `year` (int).
- Two constructors: a no-argument constructor, and one accepting all three fields.
- Getters and setters for every field.
- Declares `implements Serializable`.
- `toString()` returns a readable summary, e.g. `Car{model='C220', make='Mercedes', year=2021}`.

`CarManager` class

- Holds the data in a `List<Car>`.
- `isValidCarModel(String model)` returns `true` only if the model matches `^[A-Z0-9]+$` — uppercase letters and digits only, at least one character. Reject lowercase letters, spaces, and punctuation.
- `inputCarInfo()` reads at least three cars from the keyboard in a loop, re-prompting for `carModel` until `isValidCarModel()` returns `true` before moving on to the next field.
- `serializeCars()` writes the entire list to `cars.txt` in one `ObjectOutputStream.writeObject()` call (write the whole `List`, not one car at a time).
- `deserializeCars()` reads `cars.txt` back with `ObjectInputStream`, casts the result to `List<Car>`, and returns it as a separate list from the one built in `inputCarInfo()` — this is what lets you prove the round trip worked.
- `displayCars()` prints the list passed to it, one car per line.
- `main()` calls the methods in this order: input, serialize, deserialize, display, so the printed output demonstrably came from the file, not from memory.

Error handling

- Wrap every stream in try-with-resources.
- Catch `IOException` around file writes/reads, `ClassNotFoundException` around deserialization, and `NumberFormatException` around parsing the year from keyboard input — print a friendly message for each rather than a raw stack trace.

Exercise 2 — Club Membership Registry with Serialization

`Member` class

- Private fields: `memberID`, `memberName`, `address` (all String).
- Two constructors (no-argument and full-argument), getters/setters for all fields, `toString()`, and implements `Serializable`.

`ClubManager` class

- `memberID` format: one letter from {T, V, A}, then one branch code from {MB, MT, MN}, then 5 digits. Regex: `^[TVA](MB|MT|MN)\d{5}$`. Valid: `TMB12345`, `VMT12312`. Invalid: `TMX12345` (bad branch code), `tmb12345` (lowercase), `TMB1234` (only 4 digits).
- Ask the user how many members to enter; require a positive whole number, re-prompting otherwise, then read that many members from the keyboard.
- Read every field using `BufferedReader` wrapping `InputStreamReader(System.in)` — `Scanner` is deliberately excluded from this exercise so you practice the `java.io` API.
- Re-prompt for `memberID` until it matches the regex; `memberName` and `address` can be accepted as free text.
- Store members in a generically-typed `ArrayList<Member>`.
- Serialize the full list in one call to `member_of_club.txt` with `ObjectOutputStream`, wrapped in try-with-resources.
- Deserialize into a second, independently-declared `List<Member>` and print it so the two lists can be visually compared.

Error handling

- Wrap every stream in try-with-resources, and handle `IOException` around the file steps and `ClassNotFoundException` around deserialization with a clear message rather than a raw stack trace.

Exercise 3 — Library Book Registry with Serialization

`Book` class

- Private fields: `bookID`, `title`, `author` (all String).
- Default constructor and a 3-argument constructor; getters/setters; `toString()` returning ID, title, and author; implements `Serializable`.

`LibraryManager` class

- `bookID` format: genre letter from {F, N, S} (Fiction / Non-fiction / Science), then shift letter from {M, H} (Morning / Afternoon), then format letter from {P, D} (Paperback / Digital), then 4 digits. Regex: `^[FNS][MH][PD]\d{4}$`. Example valid: `FMP0042`. Example invalid: `XMP0042` (X isn't a genre letter). Re-prompt on failure rather than crashing or skipping the book.
- Store books in an `ArrayList<Book>`, and input `bookID`, `title`, and `author` for 5 books from the keyboard.
- Serialize the list to `library_books.ser` with `ObjectOutputStream`.
- Deserialize it back with `ObjectInputStream` into a new list and display it.
- Add `int countBooksByAuthor(List<Book> books, String author)` that iterates the deserialized list and returns how many entries match the given author (case-insensitive comparison is a reasonable default).

Error handling

- Wrap every stream in try-with-resources, and handle `IOException` around the file steps and `ClassNotFoundException` around deserialization with a clear message rather than a raw stack trace.

Exercise 4 — Employee Registry with Serialization

`Employee` class

- Private fields: `employeeID`, `fullName`, `address` (String), `age` (int).
- Default and full-argument constructors, getters/setters, `toString()`, and implements `Serializable` (required for the file steps below).

`EmployeeTest` class

- Read exactly 5 employees from the keyboard using `BufferedReader` / `InputStreamReader` (not `Scanner`), storing them in a generically-typed list.
- `employeeID` format: 2 uppercase letters, then 4 digits, then 1–2 uppercase letters. Regex: `^[A-Z]{2}\d{4}[A-Z]{1,2}$`. Valid: `AB1234C`, `XY5678D`, `ZZ0000EF`. Invalid: `A1234` (only one leading letter), `XY12345G` (5 digits), `XY123H1` (only 3 digits, and ends in a digit). Re-prompt for `employeeID` until it matches, rather than accepting or discarding an invalid entry.
- Write the list to `employee.dat` with `ObjectOutputStream`.
- Read `employee.dat` back with `ObjectInputStream` and display every employee.
- From that same deserialized list, print only the employees with `age > 25`.

Error handling

- Wrap every stream in try-with-resources, and handle `IOException` around the file steps and `ClassNotFoundException` around deserialization with a clear message rather than a raw stack trace.

Exercise 5 — Student Records from a Text File

`Student` class

- Private fields: `name` (String), `age` (int), `phone` (String).
- Default and parameterized constructors, getters/setters, and implements `Serializable`.

`input.txt` format

- The file is a simple pipe-delimited table: a header line, a line of dashes under it, then one line per student with the three fields separated by " `|` ". Example:

```
Name | Age | Phone Number
----------
John Nguyen | 25 | 0912345678
Mary Tran | 30 | 0987654321
```

Reading and validating

- Read `input.txt` with `BufferedReader`; skip the first two lines (the header and the dashes).
- For every remaining line, split on `"\|"`, trim each piece, and read them in order: `name = parts[0].trim()`, `age = Integer.parseInt(parts[1].trim())`, `phone = parts[2].trim()`.
- Wrap the age parse in a try/catch; on `NumberFormatException`, or if the parsed age isn't positive, print exactly `Not a number for that record.`
- Treat a blank `name` as invalid and print exactly `Invalid name.`; treat a blank `phone` as invalid and print exactly `Invalid phone number.`
- Write the resulting list to `Persons.obj` with `ObjectOutputStream`, one `writeObject()` call for the whole list, wrapped in try-with-resources, handling `IOException` with a clear message rather than a raw stack trace.

Exercise 6 — Personnel Management System with File Save/Load

`Employee` class

- Fields: `name` (String), `country` (String), `birthYear` (int), `salary` (int).
- No-argument and full-argument constructors, getters/setters for every field, and `toString()` — all required: without getters the sort/filter/analyze functions below can't be written, and without `toString()` (or equivalent) nothing can be displayed.
- implements `Serializable`, required for functions 5–6 below.

Menu loop

- Print the menu, read a choice, loop until option 7 (Exit) is chosen; on an out-of-range or non-numeric choice, print an error and re-show the menu rather than crashing.
- 1 — Enter employee information: prompt for name, country, birth year, and salary, and append one new `Employee` to the in-memory `ArrayList`. Require `name` and `country` to be non-blank and `salary` to be `> 0`, re-prompting for a field until it's valid.
- 2 — Sort by salary, descending: sort a copy of the list (so re-selecting other options isn't affected by a previous sort) and print it. If no employees have been entered yet, print a short message instead.
- 3 — Analyze by home region: group employees by country and print a count per country, e.g. via a `Map<String,Integer>`. If no employees have been entered yet, print a short message instead.
- 4 — Find by region + minimum salary: prompt for both, then print every employee whose country matches and whose salary is ≥ the minimum. If no employees have been entered yet, print a short message instead.
- 5 — Save to file: prompt for a filename, write the whole list with `ObjectOutputStream` (this is what "binary mode" means here — `Employee` objects, not raw bytes), wrapped in try-with-resources, and report success or failure — including a clear message if an `IOException` occurs.
- 6 — Open file and display: prompt for a filename, read it back with `ObjectInputStream` inside try-with-resources, and print every employee — report a clear error if the file doesn't exist, isn't readable, or fails with a `ClassNotFoundException`.
- 7 — Exit the program.

Exercise 7 — Social Media Account Management with File Save/Load

`Account` class

- Fields: `username`, `password`, `userType` (String), `registerYear` (Integer), `cookie` (String), `price` (Float).
- Constructors, getters/setters, `toString()`, and implements `Serializable` (needed for options 6–7).

Menu behaviour

- Show the 8-option menu on startup. Option 1 must be completed before options 2–7 can run — check whether the list is empty and print an error otherwise.
- After any option other than Exit finishes, ask: continue (`'y'` / `'Y'`), quit (`'n'` / `'N'`), or clear the screen (`'c'` / `'C'`); re-prompt on any other input.

Option details

- 1 — Enter data: ask how many accounts to enter, requiring a positive whole number (re-prompt otherwise), then read each one. Validate `0 ≤ price ≤ 100000` and that `registerYear` falls within the last 5 years — concretely, `currentYear - 5 ≤ registerYear ≤ currentYear` (get `currentYear` from `java.time.Year.now().getValue()`).
- 2 — Sort by username, alphabetically ascending, and print as a table.
- 3 — Sort by price, descending.
- 4 — Group by `userType` and print a count per type.
- 5 — Prompt for a `userType` and a minimum price; print every account of that type with price strictly greater than the minimum.
- 6 — Save to file: prompt for a filename, write the account list with `ObjectOutputStream` inside try-with-resources, and report an error — including a clear message on `IOException` — if the file can't be created.
- 7 — Open file: prompt for a filename, read it with `ObjectInputStream` inside try-with-resources, and report a clear error if it can't be opened or fails with a `ClassNotFoundException`, otherwise display the contents.
- 8 — Exit.

Exercise 8 — Library Book Records with Database Storage

`Book` class

- Fields: `id` (String), `title` (String, ≥ 5 characters), `author` (String), `publishedYear` (int, `> 1900`), and `genre` (represent as `List<String>`, e.g. novel/science/literature).

`Test` class

- Build a list of 5 books from keyboard input via `BufferedReader` (not `Scanner`). Enter a book's genres as one semicolon-separated line (e.g. `novel;classic`) and split it on `";"` to build the list.
- Validate `title` (length ≥ 5) and `publishedYear` (`> 1900`); reject and re-prompt on failure.
- Any reasonable file format is fine for this exercise — plain delimited text or Java serialization both work; pick one and be consistent. A simple delimited-text example, one book per line: `id|title|author|publishedYear|genre1;genre2`.
- Save all 5 records to `library_data.txt`, wrapping the file operations in try-with-resources and handling `IOException` (and `ClassNotFoundException`, if you choose serialization) with a clear message rather than a raw stack trace.
- Read `library_data.txt` back and print its contents.
- Search by title or author (case-insensitive substring match is a sensible default); print `NOT FOUND` if nothing matches.

Bonus

- Create a MySQL database named `library` with a `books` table whose columns match the fields above, and use a JDBC/MySQL connector to insert the full list of books into that table.

Exercise 9 — Inventory Records (File)

`Product` class

- Fields: `productId` (String), `productName` (String, ≥ 3 characters), `price` (decimal, `> 0`), `quantity` (int), and `category` (represent as `List<String>`, e.g. electronics/household/apparel).

`Test` class

- Build a list of 5 products from keyboard input via `BufferedReader` (not `Scanner`). Enter a product's categories as one semicolon-separated line (e.g. `electronics;household`) and split it on `";"` to build the list. Validate `productName`, `price`, and `quantity` (`≥ 0`), re-prompting on failure.
- Save to `inventory_data.txt` as delimited text, one product per line, e.g. `productId|productName|price|quantity|category1;category2`. Wrap the file operations in try-with-resources and handle `IOException` with a clear message rather than a raw stack trace.
- Read the file back and print every product.
- Search by name or category (case-insensitive substring match); print `NOT FOUND` if nothing matches.

Exercise 10 — Hotel Records (File)

`Hotel` class

- Fields: `hotelId` (String, format `123-ABC-456`), `hotelName` (String, ≥ 3 characters), `address` (String), `roomPrice` (decimal, `> 0`), `starRating` (int, 1–5).
- `hotelId` regex: `^\d{3}-[A-Z]{3}-\d{3}$`. Valid: `123-ABC-456`, `235-GHI-352`. Invalid: `123-abc-456` (lowercase letters).

`Test` class

- Build a list of 5 hotels from keyboard input via `BufferedReader` (not `Scanner`); re-prompt on any invalid field.
- Reject a new `hotelId` if it already exists in the list (check before adding, not after — don't just filter duplicates out later).
- Validate `hotelName`, `roomPrice`, and `starRating` in addition to the uniqueness check.
- Save to `hotel_data.txt` as delimited text, one hotel per line, e.g. `hotelId|hotelName|address|roomPrice|starRating`, then read it back and print it. Wrap the file operations in try-with-resources and handle `IOException` with a clear message rather than a raw stack trace.
- Search by name or by location (location = the `address` field). Print `NOT FOUND` if nothing matches.

Exercise 11 — Employee Records from a Formatted Text File

`Employee` class

- Fields: `fullName` (String), `salary` (int), `department` (String).
- Default and full-argument constructors, getters/setters, `toString()`, and implements `Serializable`.

`employee_data.txt` format

- The file is a pipe-delimited table: a header line, a line of dashes under it, then one line per employee with the three fields separated by " `|` ". Example:

```
Full Name | Salary | Department
----------
Jane Smith | 72000 | Engineering
Carlos Mendez | 65000 | Marketing
```

Reading and validating

- Read `employee_data.txt` with `BufferedReader`; skip the first two lines (the header and the dashes).
- For every remaining line, split on `"\|"`, trim each piece, and read them in order: `fullName = parts[0].trim()`, `salary = Integer.parseInt(parts[1].trim())`, `department = parts[2].trim()`.
- `fullName` must be non-empty; if it's blank, print exactly `Invalid name.`
- Wrap the salary parse in a try/catch; on `NumberFormatException`, or if the parsed salary isn't positive, print exactly `Salary is not a valid number for that record.`
- `department` must be non-empty; if it's blank, print exactly `Invalid department.`
- Write the resulting list to `Employees.obj` with `ObjectOutputStream`, wrapped in try-with-resources, handling `IOException` with a clear message rather than a raw stack trace.

Exercise 12 — Book Records with ISBN Validation and Database Storage

`Book` class

- Fields: `ISBN`, `title`, `author` (String), `price` (double).
- Default and full-argument constructors, getters/setters, `toString()`, and implements `Serializable` (required for the `ObjectOutputStream` step below).

`BookTest` class

- Read 5 books from the keyboard into a `List<Book>` (generics). Validate `price` (`> 0`), re-prompting on failure.
- `ISBN` format: 3 digits, a hyphen, 2 uppercase letters, a hyphen, 4 digits. Regex: `^\d{3}-[A-Z]{2}-\d{4}$`. Valid: `123-AB-4567`. Invalid: `12-ABC-456` (wrong digit/letter counts). Re-prompt on failure.
- Write the list to `books.dat` with `ObjectOutputStream`, wrapped in try-with-resources.
- Read `books.dat` back with `ObjectInputStream` inside try-with-resources, handling `IOException` and `ClassNotFoundException` with a clear message rather than a raw stack trace, and print it.
- From the deserialized list, print every book with `price < 50`.

Bonus

- Create a MySQL database named `bookmanage` with a `books` table whose columns match the fields above, and add a method to `BookTest` that inserts the full list into that table via a JDBC connector.

Exercise 13 — Movie Records with Validation and Database Storage

`Movie` class

- Fields: `movieID`, `title`, `director` (String), `rating` (double).
- Default and full-argument constructors, getters/setters, `toString()`, and implements `Serializable` (required for the file steps below).

`MovieTest` class

- Read 5 movies from the keyboard into a `List<Movie>` (generics).
- `movieID` format: 2 uppercase letters followed directly by 4 digits, with no hyphen. Regex: `^[A-Z]{2}\d{4}$`. Valid: `AB1234`. Invalid: `Ab-1234`, `ABC-123`.
- `title` must be ≥ 5 characters.
- `rating` must be a number from 1 to 5 (inclusive; decimals like 4.5 are fine since the field is a double).
- Re-prompt on any invalid field rather than discarding the whole entry.
- Write to `movies.dat` with `ObjectOutputStream` inside try-with-resources, read it back with `ObjectInputStream` inside try-with-resources, handling `IOException` and `ClassNotFoundException` with a clear message rather than a raw stack trace, and print it.
- From the deserialized list, print every movie with `rating > 4`.

Bonus

- Create a MySQL database named `moviemanage` with a `movies` table whose columns match the fields above, and add a method to `MovieTest` that inserts the full list into that table via a JDBC connector.

Exercise 14 — Customer Records with Sorting

`Customer` class

- Fields: `id` (int), `name` (String), `email` (String), `balance` (double).
- Getters/setters for all four fields, `toString()`, and implements `Serializable`.

Building the 5 customers

- For each of 5 customers, prompt the user for all four fields — `id`, `name`, `email`, and `balance` — from the keyboard, and populate the `Customer` object with them (via a constructor, setters, or both).

File and sorting

- Write the list to `customer.dat` with `ObjectOutputStream`, wrapped in try-with-resources.
- Read it back with `ObjectInputStream` inside try-with-resources, handling `IOException` and `ClassNotFoundException` with a clear message rather than a raw stack trace, and print every customer with `balance > 1000`.
- Sort the deserialized list by balance, descending, using `Collections.sort(list, Comparator.comparingDouble(Customer::getBalance).reversed())` — plain `Collections.sort(list)` won't compile unless `Customer` implements `Comparable`, which this exercise doesn't ask for, so pass a `Comparator` instead.

Exercise 15 — Cosmetic Product Records with Comparable and Serialization

`CosmeticProduct` class

- Fields: `productID` (int), `productName` (String), `price` (double).
- Getters/setters for all fields; `toString()` formatting price sensibly (e.g. `String.format("$%.2f", price)`).
- implements `Comparable<CosmeticProduct>`, with `compareTo()` implemented as `Double.compare(this.price, other.price)` for ascending order.
- implements `Serializable` too — needed for the `ObjectOutputStream` step below.

Main method

- Build a list of 3 products from keyboard input (`productID`, `productName`, `price` for each). Validate `price` (`> 0`), re-prompting on failure.
- Write all 3 to `cosmetics.dat` with `ObjectOutputStream`, using try-with-resources so the stream is guaranteed to close even if writing fails partway through.
- Read them back into a new list with `ObjectInputStream`, also inside try-with-resources, handling `IOException` and `ClassNotFoundException` with a clear message rather than a raw stack trace.
- Sort that list with `Collections.sort(list)` — this works directly because `CosmeticProduct` implements `Comparable` — and print the sorted, formatted result.

Exercise 16 — Freelancer Records with Validation

`Freelancer` class

- Fields: `id` (String), `name` (String), `hourlyRate` (float), `totalHoursWorked` (int), `skills` (`List<String>`).
- No-argument and full-argument constructors, getters/setters for all fields, and implements `Serializable` (required for the save/read steps below).

Validation rules

- `name` ≥ 3 characters.
- `hourlyRate >` $15.
- `totalHoursWorked ≥ 0`.

`Test` class

- Read data for 5 freelancers from the keyboard with `BufferedReader` (`java.io`, not `Scanner`). Enter skills as a comma-separated line and split it into a `List<String>` (e.g. `Arrays.asList(line.split(",\s*"))`).
- On a failed validation, print a specific message naming the field and re-prompt for that field rather than discarding the whole entry.
- Save the full list to `freelancers_data.dat` with `ObjectOutputStream` inside try-with-resources.
- Read `freelancers_data.dat` back inside try-with-resources, handling `IOException` and `ClassNotFoundException` with a clear message rather than a raw stack trace, and print every freelancer.
- Search by skill (case-insensitive match against each freelancer's skill list); print matches, or `NOT FOUND` if none match.
- Run this as a straightforward sequence — enter the freelancers, save, read back and display, then search — rather than a menu-driven loop.

Exercise 17 — Reading Text File Content

- Create `dulieu.txt` in your project's working directory containing exactly the text shown below (including the line breaks):

```
The java.io package contains nearly every class you might ever need to perform
input and output (I/O) in Java. All these streams represent an input source and
an output destination. The stream in the java.io package supports many data such
as primitives, Object, localized characters, etc.

A stream can be defined as a sequence of data. The InputStream is used to read
data from a source and the OutputStream is used for writing data to a
destination.

Java provides strong but flexible support for I/O related to Files and networks
but this tutorial covers very basic functionality related to streams and I/O. We
would see most commonly used example one by one:Byte Streams
```

- Write a program that opens `dulieu.txt` with `BufferedReader` wrapping `FileReader` (or, more simply, `Files.readString(Path.of("dulieu.txt"))` if you just want the whole file as one String) and prints the content to the console.
- Use try-with-resources if you use the stream-based approach, so the file handle closes automatically.

Exercise 18 — Writing and Reading Primitive Data with Streams

Program 1 — write

- Declare one variable each of type `int`, `double`, `boolean`, `char`, and `String`; prompt the user for each from the keyboard.
- Open `data.dat` with `DataOutputStream`, wrapped in try-with-resources, and write the five values in this order, using the matching method for each type: `writeInt()`, `writeDouble()`, `writeBoolean()`, `writeChar()`, then `writeUTF()` for the String. Handle `IOException` with a clear message rather than a raw stack trace.

Program 2 — read

- Open `data.dat` with `DataInputStream`, wrapped in try-with-resources, and read the five values back in the same order they were written: `readInt()`, `readDouble()`, `readBoolean()`, `readChar()`, then `readUTF()`. Handle `IOException` with a clear message rather than a raw stack trace.
- Print all five values to the console.

Exercise 19 — Book Registry with Text File I/O

`Book` class

- Fields: `isbn`, `bookName`, `author`, `publisher` (String), `price` (float).
- Full set of constructors, getters/setters, and `toString()`.

`BookManager` class

- Prompt the user for how many books to enter (`n`) — require a positive whole number, re-prompting otherwise — then read all `n` using `BufferedReader` (not `Scanner`).
- Write the books to `book.txt` as delimited text, one line per book, e.g. `isbn|bookName|author|publisher|price` — use a plain `BufferedWriter` / `PrintWriter`, not `ObjectOutputStream`: the goal here is human-readable text output. Wrap the writer in try-with-resources and handle `IOException` with a clear message rather than a raw stack trace.

`BookRead` class

- Open `book.txt` with `BufferedReader` inside try-with-resources, split each line on `|` to reconstruct a `Book` object per line, and collect them into a list, handling `IOException` with a clear message rather than a raw stack trace.
- Print the full list.
- Prompt for a title and print the matching book(s), or print `NOT FOUND` if none match.